Predicting Falcon 9 first-stage landing success

Applied Data Science Capstone

Joao Trevisan
23 October 2026

Submitted public GitHub repository URL

https://github.com/joaoltrevisan/spacex-falcon9-capstone

The repository contains executed notebooks, Python source, laboratory datasets, interactive maps and the final report. All links in this PDF are clickable.

Executive summary

- Objective: estimate whether the first stage lands successfully, supporting an initial assessment of recovery potential.
- The 90-row modeling sample contains 60 successful landings (66.7%). Site, orbit, payload and mission sequence show descriptive associations.
- Decision Tree wins training cross-validation (87.7%) and achieves 83.3% on the 18-row holdout: 15 correct predictions.
- The findings describe historical samples. They do not establish causal effects or estimate actual launch costs.

Introduction and business question

- First-stage reuse is relevant to launch economics. The prediction target is successful landing, not payload delivery or total mission success.
- Question: how do launch characteristics relate to landing outcomes, and which classifier performs best on the provided dataset?
- Workflow: collect historical data → validate and encode → explore with charts and SQL → inspect maps/dashboard → tune classifiers → evaluate holdout.
- A technical audience can reproduce the analysis through the linked notebooks and source files.

Data sources and sample boundaries

- API collection: 107 archived launch records; web scraping: 97 historical launch rows from Wikipedia revision 987426997.
- Modeling and six EDA charts: the supplied 90 × 80 encoded matrix, aligned against the IBM 90-row labeled table.
- SQL: the separate 101-row Spacex.csv laboratory dataset. Folium and Dash: their separate 56-record laboratory datasets.
- These samples are not joined or treated as interchangeable. Counts, categories and success rates are reported with their own denominator.

API collection: archived response to normalized table

- Read IBM’s archived API_call_spacex_api.json → verify a JSON list → flatten launch and first-core fields → sort by flight number → export CSV.
- Extract dates, rocket/launchpad IDs, payload IDs, core reuse fields and landing outcomes. Preserve nested payload/failure data as JSON.
- Output: 107 rows × 18 columns. The notebook displays the resulting records and counts by rocket ID.
- This is reproducible archived-response processing. The separate supplied modeling table provides enriched features; live API enrichment is not claimed.

Executed notebook / source: notebooks/01_Data_Collection_API.ipynb

Web scraping: fixed historical Wikipedia revision

- Read saved HTML for revision 987426997 → identify flight/date tables → flatten headers and remove reference markers.
- Parse UTC dates and numeric flight identifiers → retain 2010-06-04 through 2020-11-05 → sort and deduplicate by flight number.
- Output: 97 historical launch rows. The executed notebook displays sample rows, date coverage and duplicate count.
- The historical revision avoids silently mixing the present-day launch list with the course's older sample.

Executed notebook / source: notebooks/02_Data_Collection_Web_Scraping.ipynb

Wrangling and target definition

- Validate 90 feature rows against the labeled course table using flight order and decoded Orbit, LaunchSite, LandingPad and Serial categories.
- Retain the course’s prepared numeric values; missing landing-pad values are represented as Unknown in the display table.
- Class 1: successful landing. Class 0: the remaining landing outcomes, including no landing attempt or unavailable outcome.
- Recreate one-hot encoding, cast to float64 and verify all 80 columns match the supplied matrix. Labels: 60 positive, 30 negative.

Executed notebook / source: notebooks/03_Data_Wrangling_and_Target.ipynb

EDA methods: six complementary views

- Scatter plots: flight number versus site; payload versus site; flight number versus orbit; payload versus orbit. Color indicates landing outcome.
- Bar chart: mean success rate by orbit, annotated with the number of observations.
- Line chart: annual mean success rate, with yearly sample sizes.
- Purpose: identify descriptive patterns, differences in mission mix and sparse categories before interpreting model performance.

Executed notebook / source: notebooks/04_EDA_and_Feature_Engineering.ipynb

SQL methods: ten laboratory questions

- Load Spacex.csv into SQLite; trim surrounding whitespace in text fields; retain 101 dated records.
- Use DISTINCT, LIKE, aggregates, MIN/MAX, grouped counts, a maximum-payload subquery and DENSE_RANK.
- Answer all ten template questions. NASA CRS is reported separately from all customer strings mentioning NASA.
- Mission_Outcome and Landing_Outcome remain separate: payload mission success is not equivalent to successful booster recovery.

Executed notebook / source: notebooks/05_SQL_EDA_and_Folium_Map.ipynb

Folium methods: locations, outcomes and proximity

- Markers and 1 km circles identify sites; a MarkerCluster preserves all 56 launch records and expands overlapping points.
- Green markers represent success; red markers represent failure. Popups identify individual flights.
- Two PolyLines connect CCAFS LC-40 to a visually selected coastline point and Samuel C. Phillips Parkway; Haversine measures straight-line distance.
- MousePosition supplies the coordinates selected on OpenStreetMap. Distances are approximate straight-line measurements, not surveyed nearest distances.

Executed notebook / source: src/build_folium_map.py

Dash methods: interactive comparison

- All-site pie: successful launch counts by site, totaling 24. Selected-site pie: success/failure proportions within that site.
- Scatter: payload versus binary landing outcome; color identifies booster version; hover exposes flight and site.
- Dropdown filters launch site. RangeSlider filters scatter payload from 0 to 10,000 kg; the sample count updates with both filters.
- The pie uses all payloads at the selected site. Its denominator is explicitly different from the filtered scatter denominator.

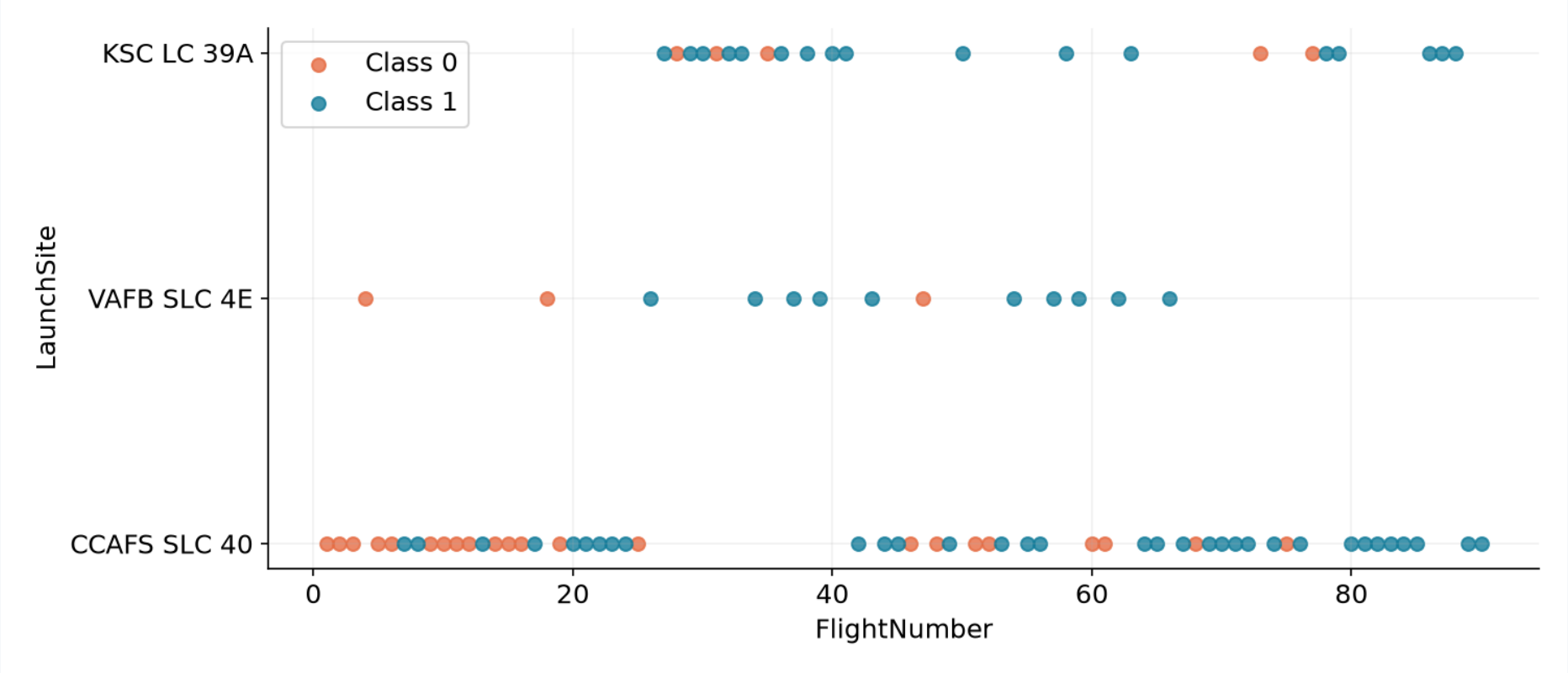
Executed notebook / source: src/dash_app.py

Classification: train, tune and evaluate

- Stratified split, seed 2: 72 training records and 18 holdout records. Five-fold stratified cross-validation operates on training data.
- Compare Logistic Regression, SVM, KNN and Decision Tree with GridSearchCV. Scaling is inside the training pipelines.
- Select by cross-validation accuracy, then report holdout accuracy, precision, recall, F1 and confusion matrix.
- Limitations: only 18 holdout rows; correlated flight/date/booster features; random split does not demonstrate forecasting performance on future launches.

Executed notebook / source: notebooks/07_Modeling.ipynb

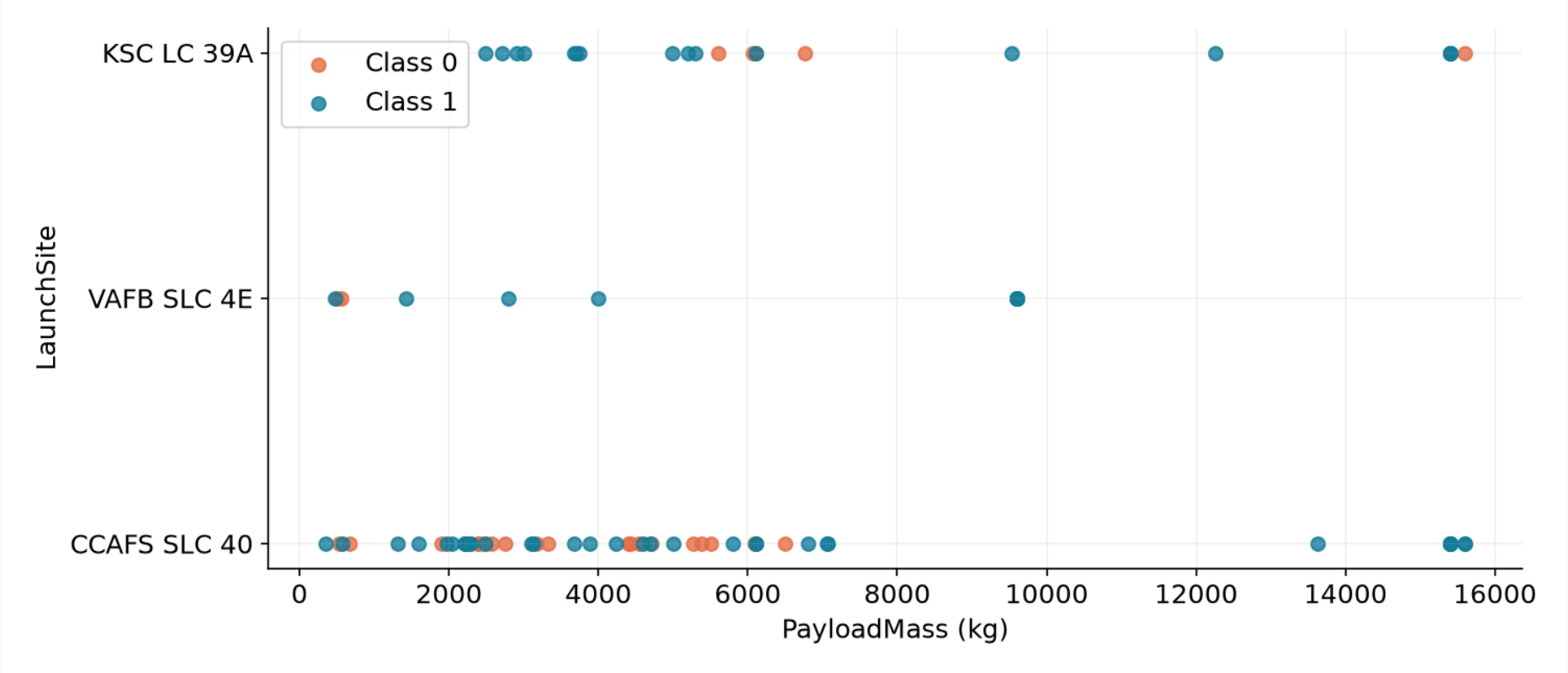
Flight number versus launch site



Later missions show more successes. Site comparisons also reflect different mission periods and objectives.

Source: notebooks/04_EDA_and_Feature_Engineering.ipynb

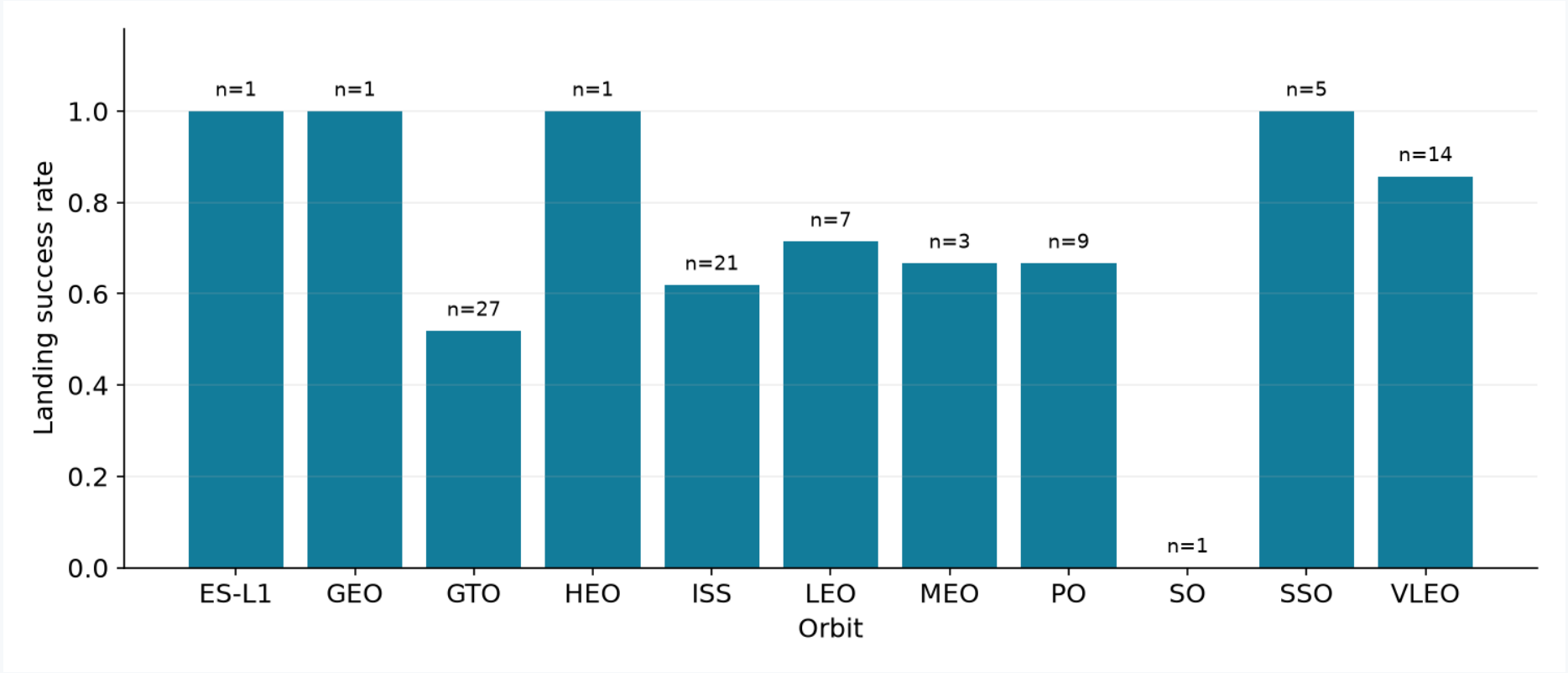
Payload versus launch site



Both outcomes occur across payload levels. CCAFS SLC 40 has 55 records, KSC 22 and VAFB 13; the sites have different mission mixes.

Source: notebooks/04_EDA_and_Feature_Engineering.ipynb

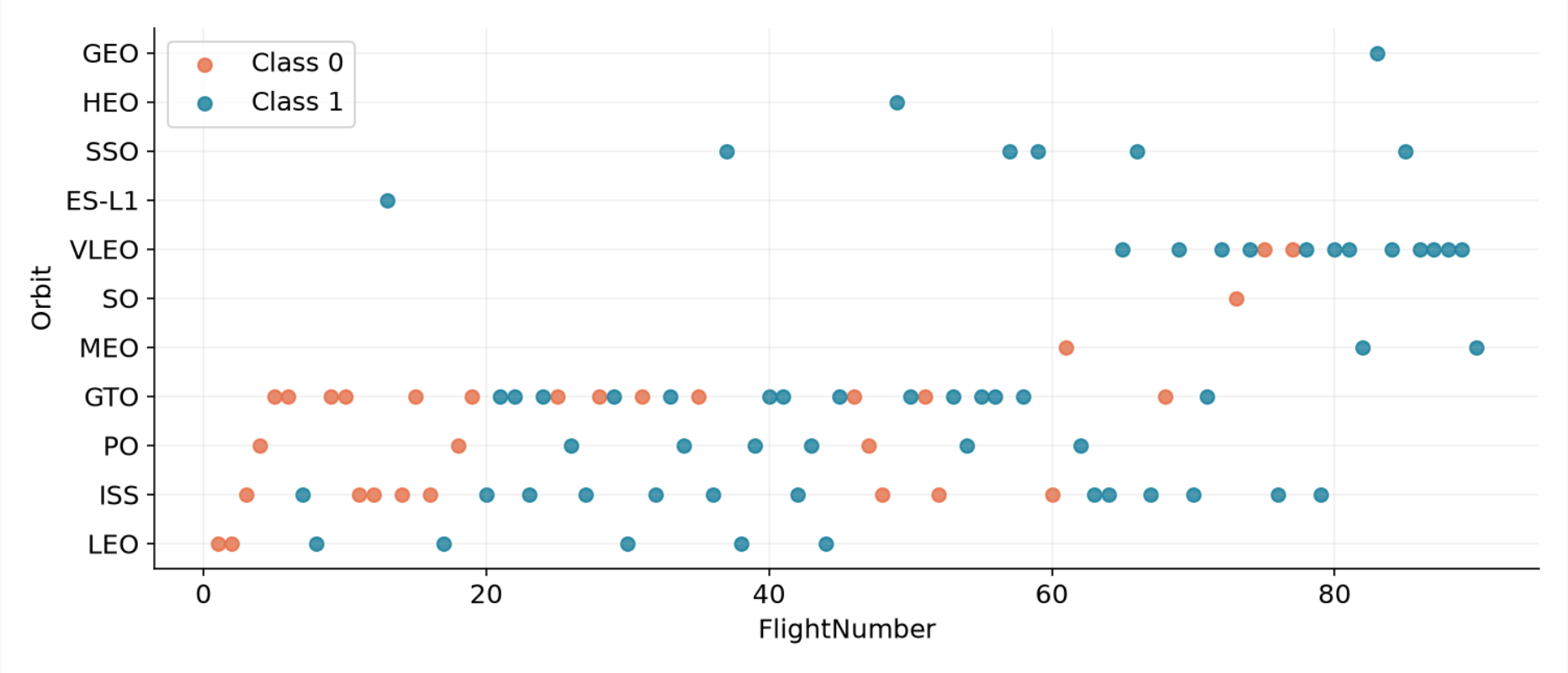
Landing success by orbit



SSO is 5/5; VLEO 12/14; GTO 14/27. A 100% rate based on a single flight is weak evidence of general reliability.

Source: notebooks/04_EDA_and_Feature_Engineering.ipynb

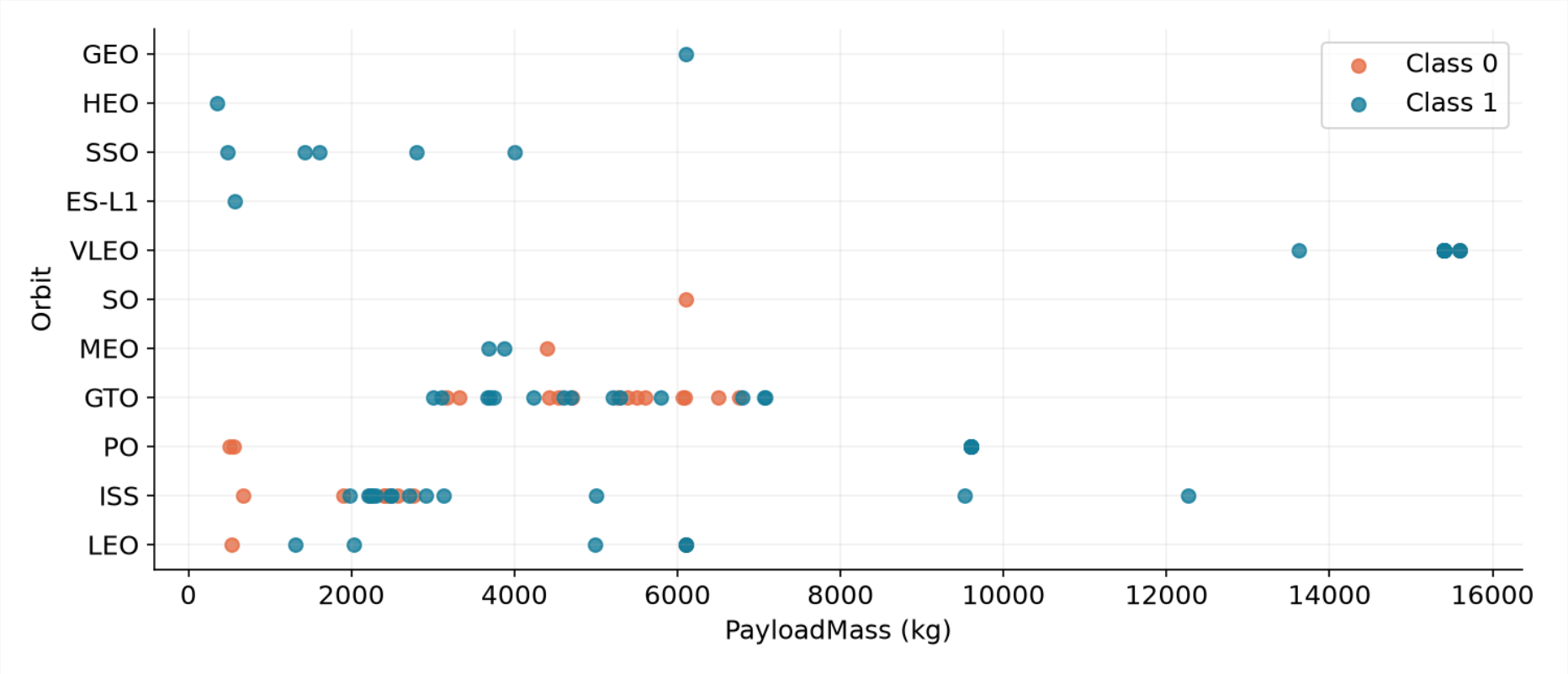
Flight number versus orbit



Orbit categories are unevenly distributed over mission sequence. Apparent orbit effects can be confounded with experience and hardware changes.

Source: notebooks/04_EDA_and_Feature_Engineering.ipynb

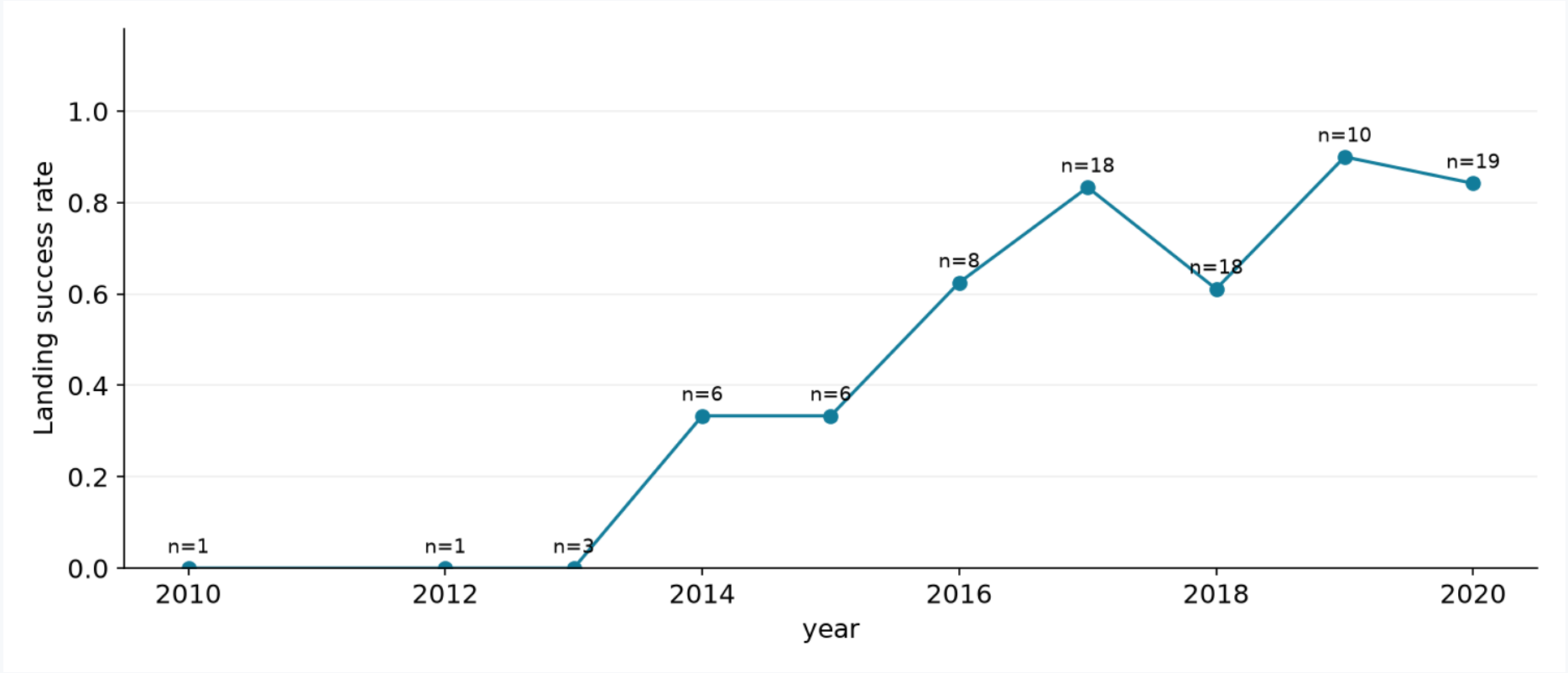
Payload versus orbit



Payload and orbit are associated with mission requirements. The plot does not establish that increasing payload improves landing success.

Source: notebooks/04_EDA_and_Feature_Engineering.ipynb

Annual landing success



Success rises from 0/1 in 2010 to 16/19 in 2020, with a decline to 11/18 in 2018. Early years have very small samples.

Source: notebooks/04_EDA_and_Feature_Engineering.ipynb

SQL 01 | Unique launch sites

Launch Site
CCAFS LC-40
CCAFS SLC-40
KSC LC-39A
VAFB SLC-4E

Four source site labels; LC-40 and SLC-40 are preserved as recorded.

Executed SQL

```
SELECT DISTINCT Launch_Site FROM launches ORDER BY Launch_Site
```

Executed SQL notebook

SQL 02 | First five CCA launch-site records

Date	Booster Version	Launch Site	PAYLOAD MASS KG
2010-06-04	F9 v1.0 B0003	CCAFS LC-40	0
2010-12-08	F9 v1.0 B0004	CCAFS LC-40	0
2012-05-22	F9 v1.0 B0005	CCAFS LC-40	525
2012-10-08	F9 v1.0 B0006	CCAFS LC-40	500
2013-03-01	F9 v1.0 B0007	CCAFS LC-40	677

The first five chronological CCA records are from 2010 to 2013.

Executed SQL

```
SELECT Date, Booster_Version, Launch_Site, PAYLOAD_MASS__KG_ FROM launches WHERE Launch_Site LIKE 'CCA%' ORDER BY Date LIMIT 5
```

Executed SQL notebook

SQL 03 | Total payload: NASA CRS

Customer	payload kg
NASA (CRS)	45596

NASA (CRS): 45,596 kg. All strings mentioning NASA: 107,010 kg across 32 records. The filters answer different questions.

Executed SQL

```
SELECT Customer, SUM(PAYLOAD_MASS__KG_) AS payload_kg FROM launches WHERE Customer = 'NASA (CRS)' GROUP BY Customer
```

Executed SQL notebook

SQL 04 | Average payload: F9 v1.1

launches	mean payload kg
15	2534.67

Mean = 2,534.67 kg across 15 F9 v1.1 records.

Executed SQL

```
SELECT COUNT(*) AS launches, ROUND(AVG(PAYLOAD_MASS__KG_), 2) AS mean_payload_kg FROM launches WHERE Booster_Version LIKE 'F9 v1.1%'
```

Executed SQL notebook

SQL 05 | First successful ground-pad landing

first successful ground landing
2015-12-22

The first ground-pad success in this dataset is 22 December 2015.

Executed SQL

```
SELECT MIN(Date) AS first_successful_ground_landing FROM launches WHERE Landing_Outcome = 'Success (ground pad)'
```

Executed SQL notebook

SQL 06 | Droneship success: 4,000 < payload < 6,000 kg

Date	Booster Version	PAYLOAD MASS KG
2016-05-06	F9 FT B1022	4696
2016-08-14	F9 FT B1026	4600
2017-03-30	F9 FT B1021.2	5300
2017-10-11	F9 FT B1031.2	5200

Four missions meet both the successful droneship outcome and strict payload bounds.

Executed SQL

```
SELECT Date, Booster_Version, PAYLOAD_MASS__KG_ FROM launches WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ < 6000 ORDER BY Date
```

Executed SQL notebook

SQL 07 | Mission outcome counts

Mission Outcome	missions
Success	99
Success (payload status unclear)	1
Failure (in flight)	1

99 Success + 1 Success (payload status unclear) + 1 Failure (in flight). These are mission outcomes, not landing labels.

Executed SQL

```
SELECT Mission_Outcome, COUNT(*) AS missions FROM launches GROUP BY Mission_Outcome ORDER BY missions DESC
```

Executed SQL notebook

SQL 08 | Boosters carrying the maximum payload

Booster Version	PAYLOAD MASS KG
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

The maximum is 15,600 kg, shared by 12 listed booster records.

Executed SQL

```
SELECT Booster_Version, PAYLOAD_MASS__KG_ FROM launches WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM launches) ORDER BY Date
```

Executed SQL notebook

SQL 09 | 2015 droneship landing failures

Date	month	Booster Version	Launch Site	Landing Outcome
2015-01-10	01	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
2015-04-14	04	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

Two failures: January and April 2015, both at CCAFS LC-40.

Executed SQL

```
SELECT Date, substr(Date,6,2) AS month, Booster_Version, Launch_Site, Landing_Outcome FROM launches WHERE Date LIKE '2015-%' AND Landing_Outcome = 'Failure (drone ship)' ORDER BY Date
```

Executed SQL notebook

SQL 10 | Rank landing outcomes within the date window

Landing Outcome	launches	rank
No attempt	10	1
Failure (drone ship)	5	2
Success (drone ship)	5	2
Controlled (ocean)	3	3
Success (ground pad)	3	3
Failure (parachute)	2	4
Uncontrolled (ocean)	2	4
Precluded (drone ship)	1	5

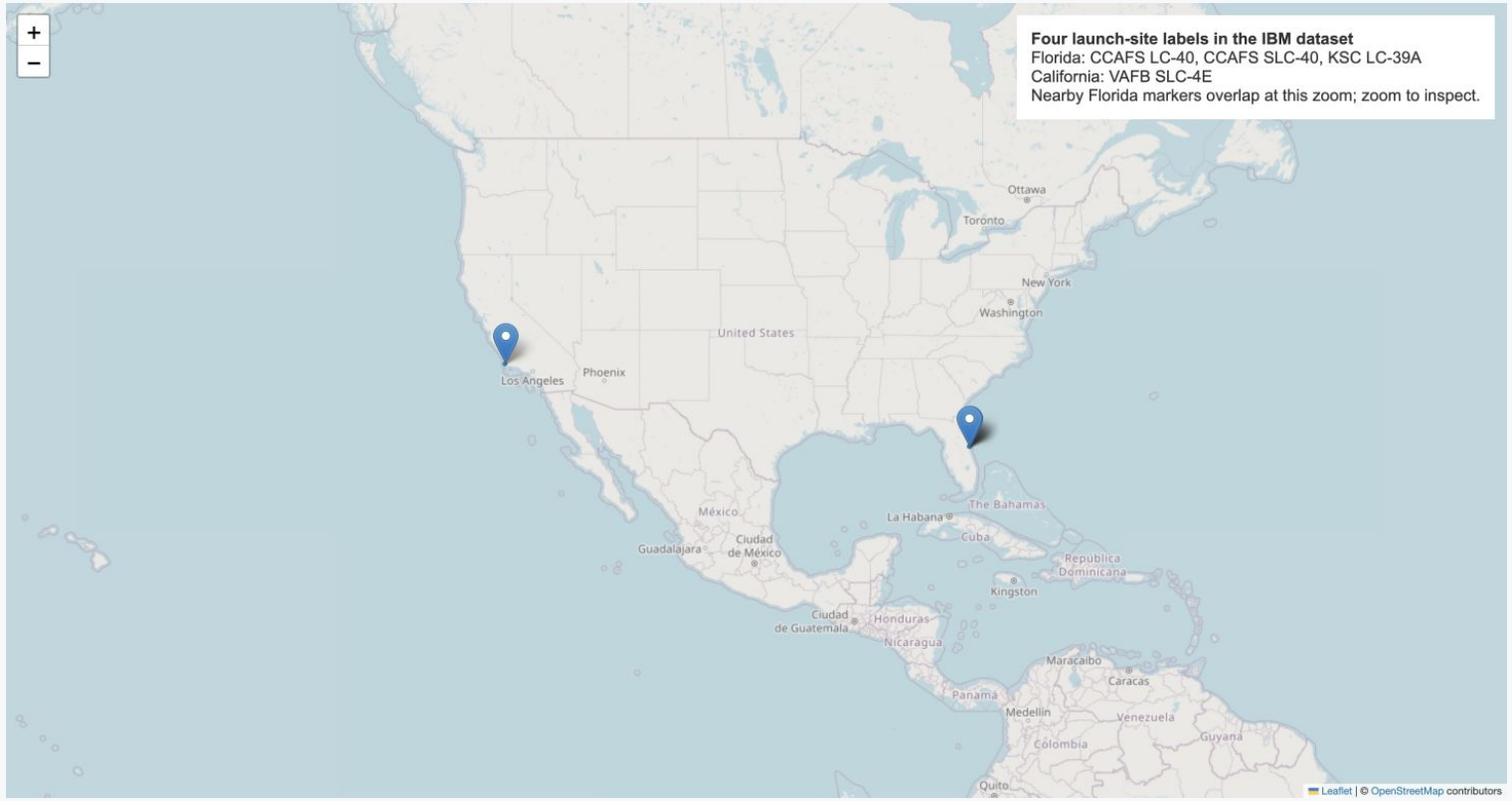
Inclusive dates: 2010-06-04 through 2017-03-20. Ties share a dense rank.

Executed SQL

```
SELECT Landing_Outcome, COUNT(*) AS launches, DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS rank FROM launches WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY Landing_Outcome ORDER BY launches DESC, Landing_Outcome
```

Executed SQL notebook

Folium | all launch-site locations



Four source labels span Florida and California. Three Florida markers overlap at national zoom; interactive zoom and popups distinguish them.

Source: src/build_folium_map.py

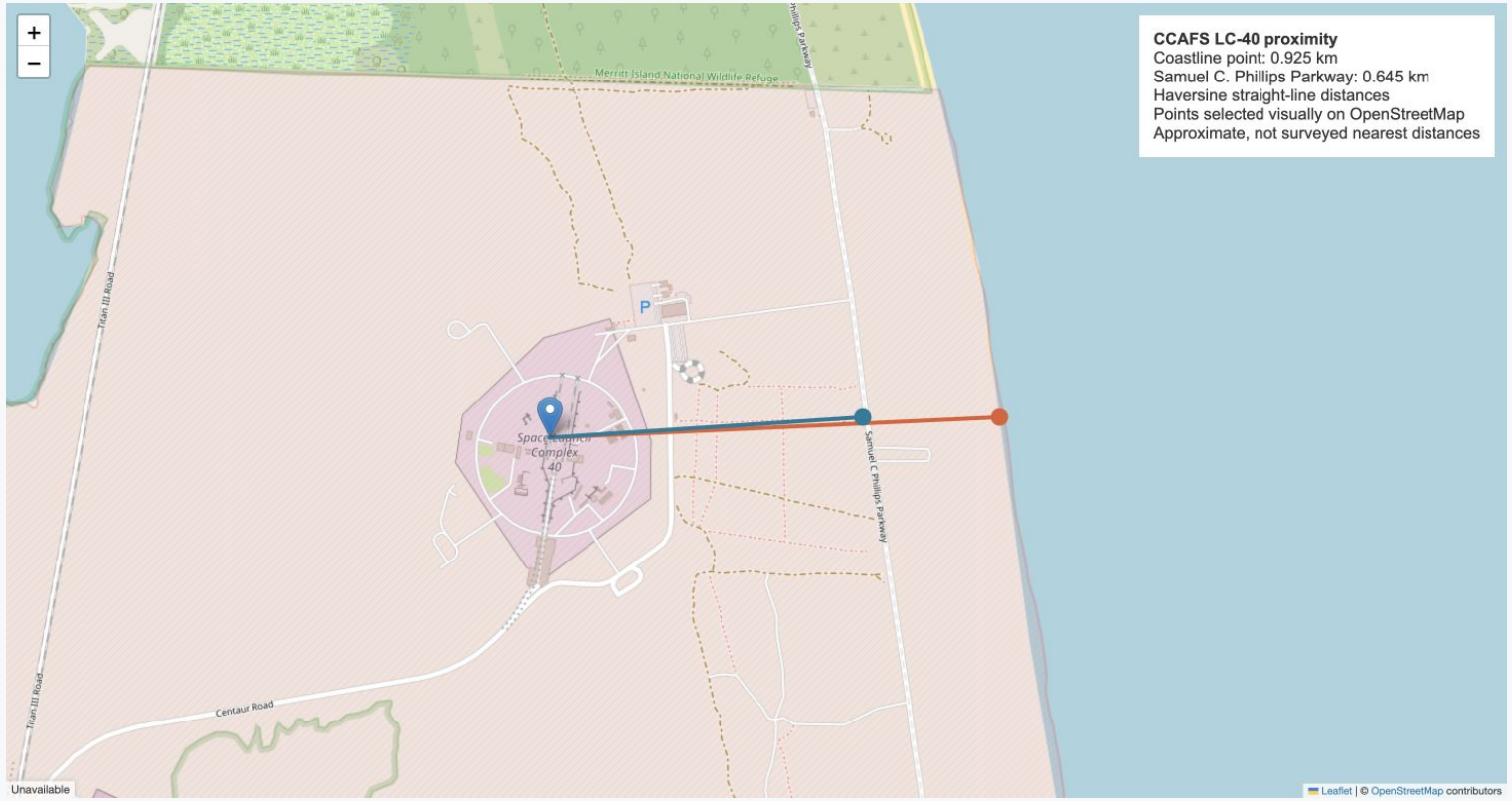
Folium | color-coded launch outcomes



Expanded KSC cluster: 13 records, including 10 successes (green) and 3 failures (red). Spiderfied markers separate overlapping coordinates.

Source: src/build_folium_map.py

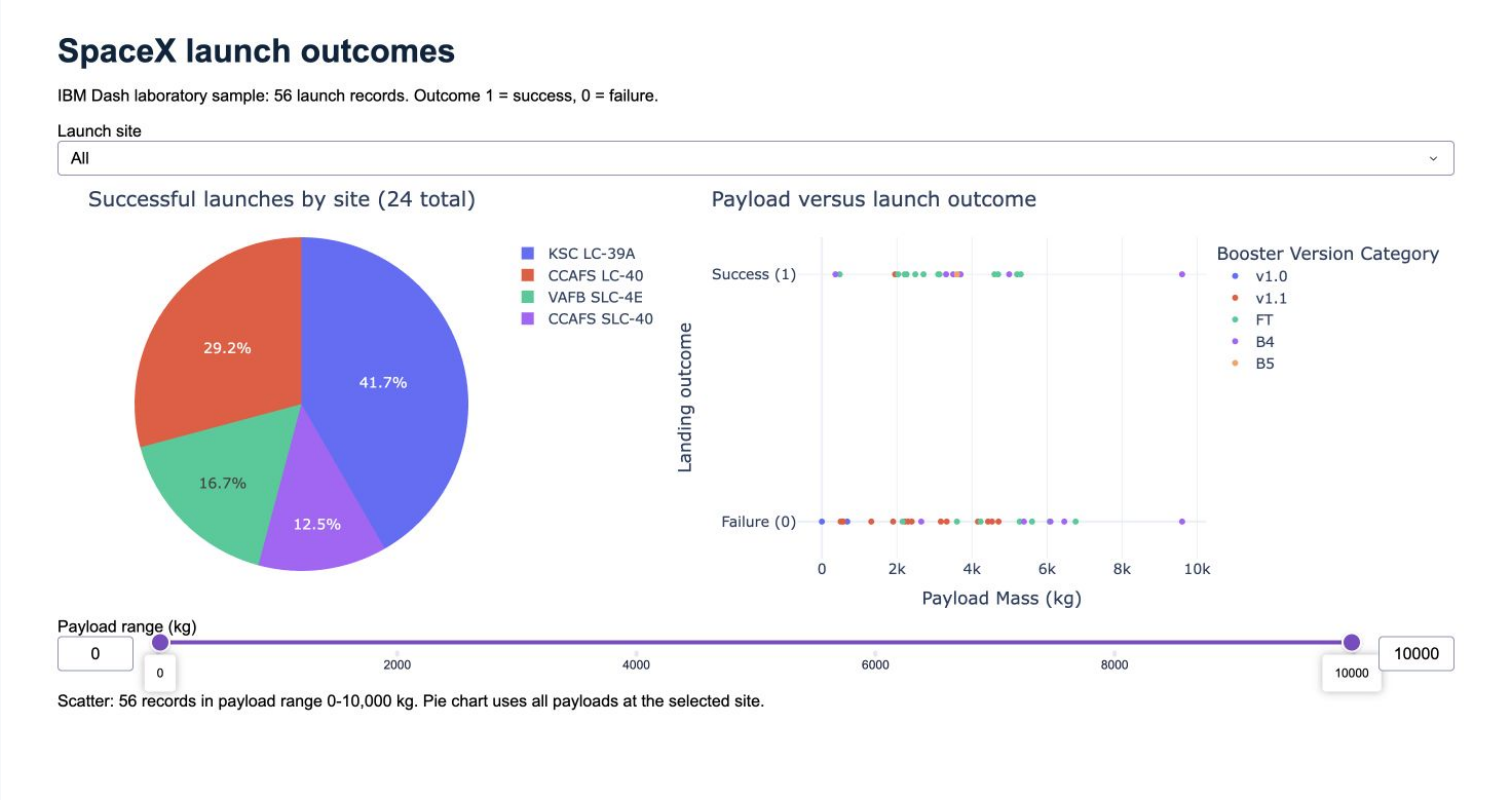
Folium | proximity to coastline and roadway



CCAFS LC-40 to the selected coastline point: 0.925 km; to Samuel C. Phillips Parkway: 0.645 km. Coordinates were read visually with MousePosition; distances are approximate.

Source: src/build_folium_map.py

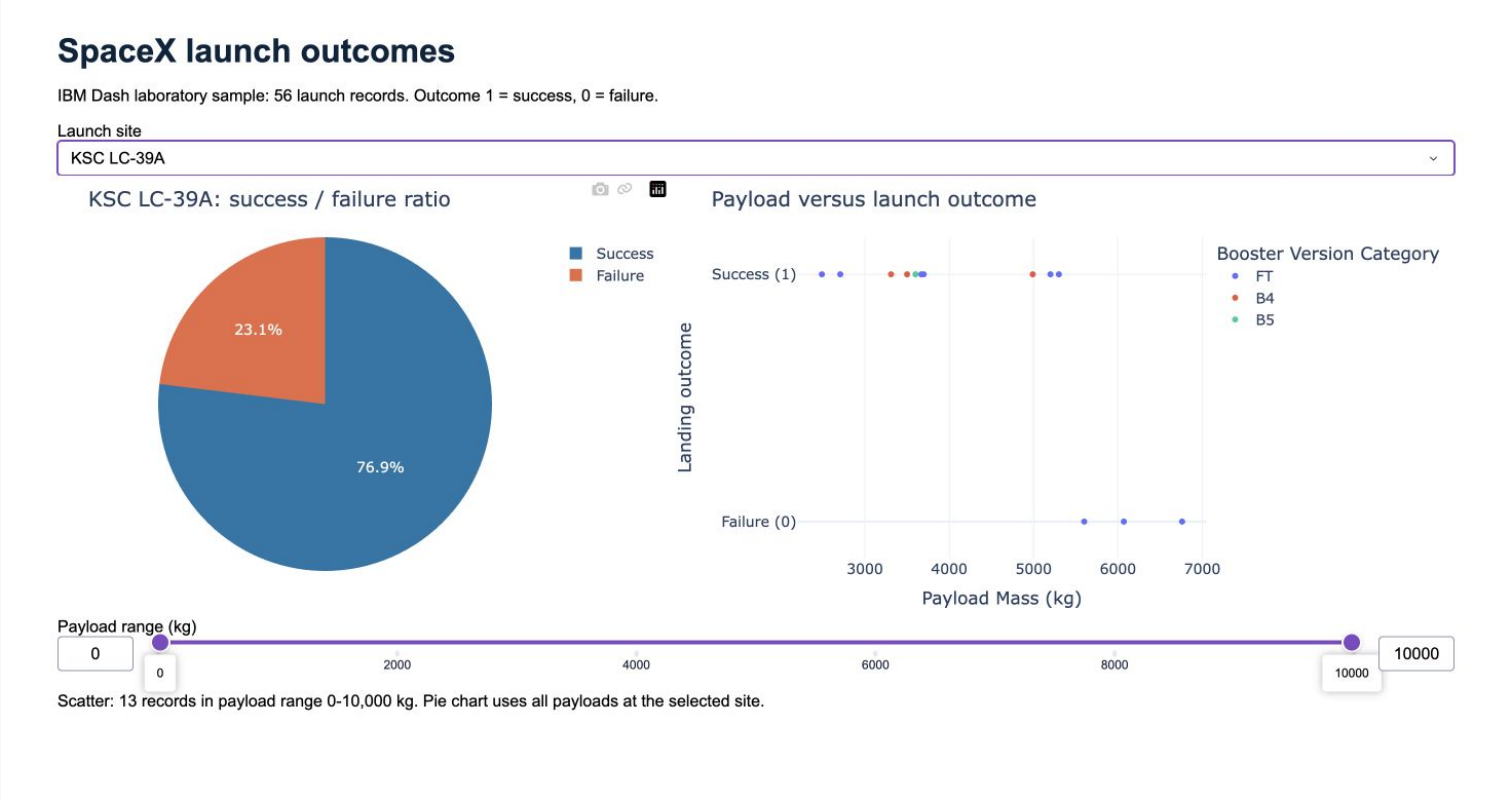
Dash | successful launches across all sites



KSC contributes 10/24 successes (41.7% of successful launches); CCAFS LC-40 7, VAFB 4, CCAFS SLC-40 3. This pie shows counts, not per-site success rates.

Source: notebooks/06_Plotly_Dash_Dashboard.ipynb

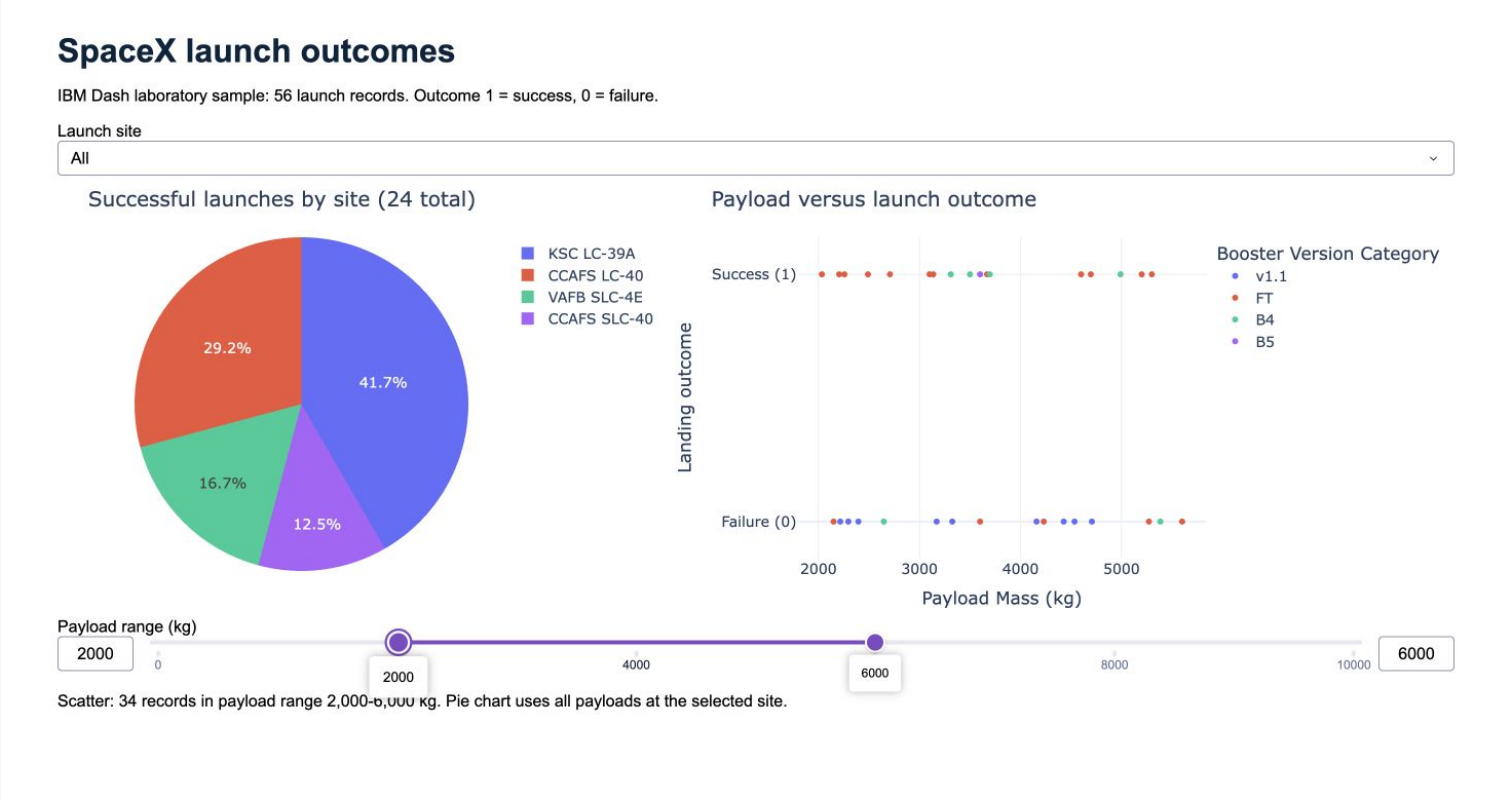
Dash | highest site success ratio



KSC: 10/13 = 76.9%, the highest ratio in this 56-record sample. Other rates: CCAFS LC-40 26.9%, SLC-40 42.9%, VAFB 40.0%.

Source: notebooks/06_Plotly_Dash_Dashboard.ipynb

Dash | payload filter changes the sample



Changing All sites from 0-10,000 kg (56 rows) to 2,000-6,000 kg retains 34 rows. Both outcomes occur within the filtered interval.

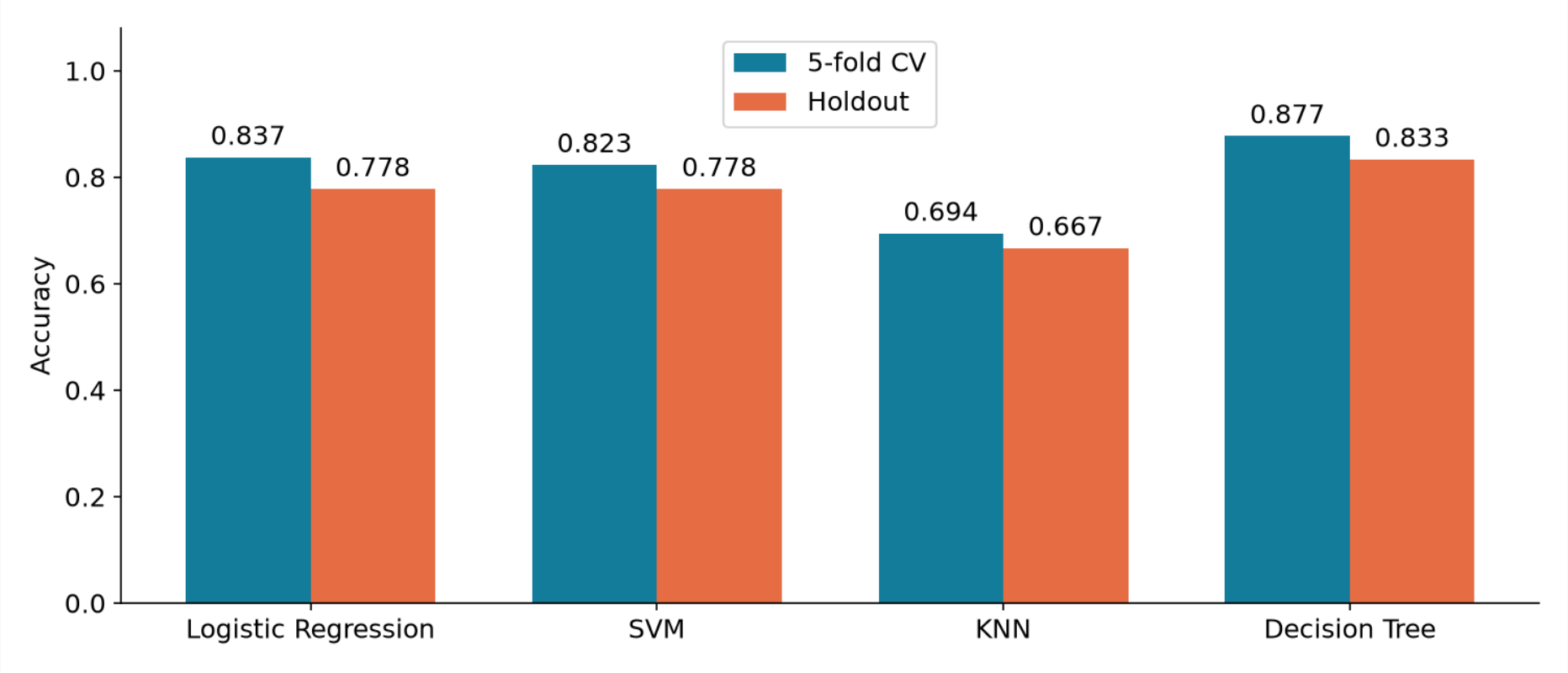
Source: notebooks/06_Plotly_Dash_Dashboard.ipynb

Dashboard findings: report denominators

- Filtered 2,000-6,000 kg sample: 18 successes / 34 launches (52.9%).
- Booster categories: B4: 6/11 (54.5%); B5: 1/1 (100.0%); FT: 16/24 (66.7%); v1.0: 0/5 (0.0%); v1.1: 1/15 (6.7%).
- The best observed category rate can be based on few launches; hardware version is also associated with mission period.
- Counts and ratios answer different questions: KSC has the largest success count and highest site ratio here, but that does not isolate a causal site advantage.

Executed notebook / source: notebooks/06_Plotly_Dash_Dashboard.ipynb

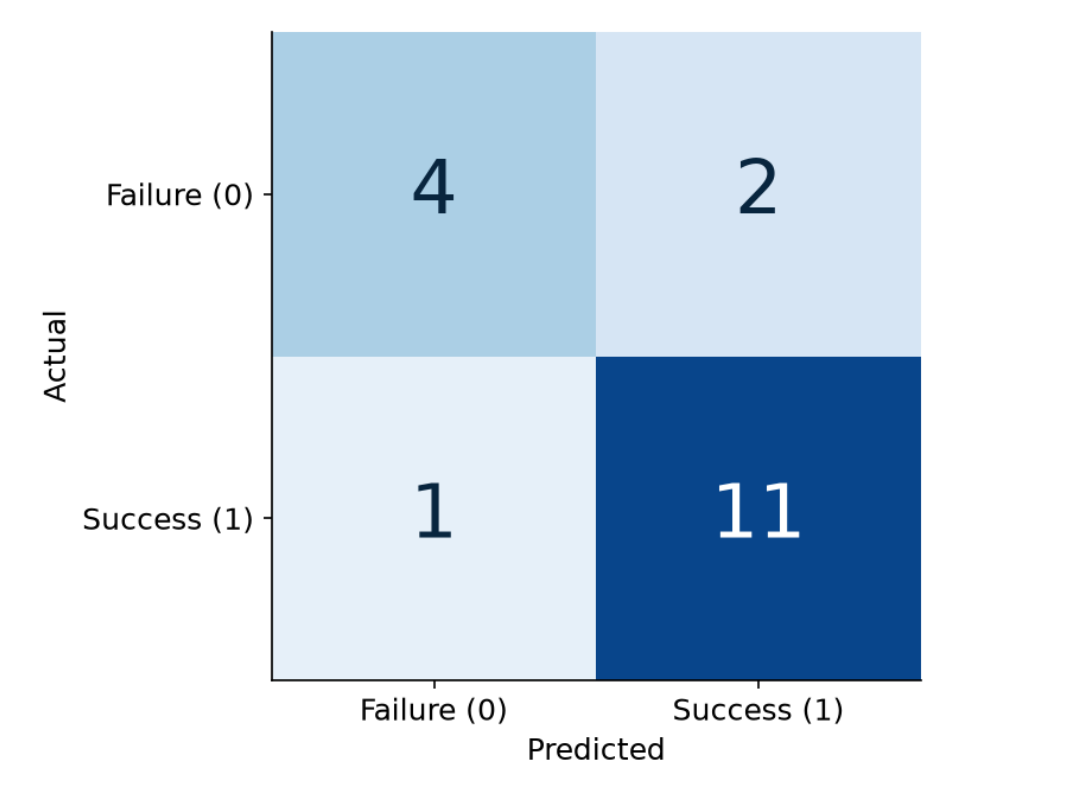
Model comparison: cross-validation and holdout



Decision Tree is selected by CV (0.877); holdout accuracy is 0.833. KNN predicts all holdout rows as success and matches the majority-class baseline.

Source: notebooks/07_Modeling.ipynb

Decision Tree confusion matrix



Holdout: TN=4, FP=2, FN=1, TP=11. Recall=11/12=91.7%; precision=11/13=84.6%; F1=0.880. Three errors among 18 predictions.

Source: notebooks/07_Modeling.ipynb

Selected hyperparameters and validation limits

- Logistic Regression: C=1. SVM: linear kernel, C=0.1. KNN: 3 neighbors, uniform weights.
- Decision Tree: gini, maximum depth 5, minimum samples per leaf 1. Selected by training cross-validation.
- One changed prediction shifts holdout accuracy by 5.6 percentage points. The observed model ranking needs validation on a larger independent sample.
- Next experiment: chronological evaluation, remove features unavailable before launch, and assess calibration before using probabilities for operational decisions.

Executed notebook / source: notebooks/07_Modeling.ipynb

Conclusions

- The historical 90-row sample has a 66.7% landing-success baseline. Later missions generally show higher success, with substantial variation by mission mix.
- The 56-row dashboard sample yields a different baseline: 24/56 = 42.9%. Mixing these denominators would create misleading conclusions.
- Decision Tree achieves 15/18 correct holdout predictions, exceeding the 12/18 majority baseline by three predictions.
- Geographic proximity and descriptive associations support exploration; they do not prove why a landing succeeds. More data and temporal validation are required.

APPENDIX

Reproducibility and source references

notebooks/01_Data_Collection_API.ipynb

notebooks/02_Data_Collection_Web_Scraping.ipynb

notebooks/03_Data_Wrangling_and_Target.ipynb

notebooks/04_EDA_and_Feature_Engineering.ipynb

notebooks/05_SQL_EDA_and_Folium_Map.ipynb

notebooks/06_Plotly_Dash_Dashboard.ipynb

notebooks/07_Modeling.ipynb

Run: pip install -r requirements.txt; python src/sql_lab.py; python src/build_folium_map.py; python src/analyze_model.py; python src/dash_app.py

Official IBM presentation template

Official submission overview